

0.76.11. Un déclencheur en (0, 0) : le carré plus rapide — en simple

Le 0.76.3 en simple : un seul nombre change, la vitesse de tourne_carre, 250 → 80.

1. Ce qu'il faut comprendre

C'est le 0.76.3, en simple : la vitesse, 6e réglage de `tourne_carre`, passe de `250` à `80` millisecondes.

80 ms, c'est 5 images : exactement la vitesse du 0.76.3 (`bimages == 5`). Mais ici, on la dit en millisecondes, comme partout ailleurs.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)
- `#include <tourne_carre>` → [0.76.9. La fonction `tourne\_carre\(\)` — un carré sans arrêter le jeu](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1695 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	75 o	← la console, à l'allumage – la mise en route de la console
tourne_carre()	410 o	← main() – une fonction fournie par le langage
deplace_croix()	217 o	← main() – une fonction fournie par le langage
main()	101 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code>	× 3	39 o sur place, dans <code>tourne_carre()</code> , <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o)
<code>effacer()</code>	× 2	30 o sur place, dans <code>tourne_carre()</code> , <code>deplace_croix()</code> + <code>AdresseCase</code> (10 o), <code>EffacerCases</code> (12 o)
<code>bouton()</code>	× 4	20 o sur place, dans <code>deplace_croix()</code> + <code>LireManette</code> (37 o)
<code>nombre()</code>	× 2	20 o sur place, dans <code>main()</code> + <code>EcrireNombre</code> (33 o)
<code>images()</code>	× 2	4 o sur place, dans <code>tourne_carre()</code> , <code>deplace_croix()</code>

```
image()          × 1      3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
tourne_carre()   × 1      26 o sur place, dans main()
deplace_croix() × 1      19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>        // écrit un nombre en chiffres
#include <tourne_carre>  // une tuile qui tourne en carré sans arrêter le jeu
```

2. UNE VARIABLE — X

```
uint8_t x = 10; // le A part de (10, 0)
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

4. UNE VARIABLE — ACTIF

```
uint8_t actif = 0; // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position
        nombre(4, 17, y);

        // Le déclencheur, en simple : pas besoin de « && actif == 0 ».
        // Remettre actif à 1 quand il vaut déjà 1 ne change rien.
        if (x == 0 && y == 0) {
            actif = 1;
        }

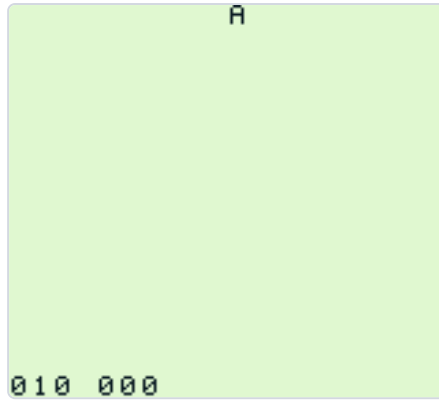
        // Tout ce qui se passe après le déclencheur, une ligne par lettre :
        if (actif == 1) {
            tourne_carre(0, 10, 8, ALPHABET[1], 4, 80); // le B tourne en carré
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.76.3 : le carré tourne trois fois plus vite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en 150 images, un tour entier

Ensuite : **0.76.12. Un déclencheur en (0, 0) : deux B — en simple** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++